Which model should read what comes from outside

The live setup scores every text from outside with Horizon-Labs' classifier and laya:en, and lets Claude Sonnet decide everything either finds suspect. It catches 83%, 94% and 79% of the injections in three held-out test sets, the last one hidden inside tool output, and wrongly flags 7 of our 240 own texts. A clean outside read costs about a quarter of a second; roughly 40% of them wait about 2 seconds more for Claude.

Scoring server: Apple M5 Pro, 24 GB, on the Mac that hosts the Windows VM. Hook: Windows 11 ARM64 VM, 8 GB. Models run locally; only Claude, which already reads this session, sees text off the machine.

AT A GLANCE

94%	79%	2.9%	0.26 s
of xTRam1 test injections caught by the live setup (Horizon-Labs alone: 56%)	of injections hidden inside tool output caught, on AgentInjectionBench (Horizon-Labs alone: 65%)	of our own tickets, notes, Finnish UI strings and READMEs wrongly flagged (7 of 240)	added to a clean outside read by the whole hook; about 2 s more when Claude has to decide

Patterns and secrets
every call, in the hook
→
Horizon-Labs and laya:en
every outside text, about 70 ms
→
Claude Sonnet, no thinking
if either scores 0.5 or more, about 2 s
→
Note to Claude, log, toast
toast only when confirmed

HOW IT WAS MEASURED

Held-out attacks, and our own text for false alarms

Attacks came from two published labelled sets, scored as they are: the 116-row test split of **deepset/prompt-injections** (60 injections) and the first 800 rows of the **xTRam1/safe-guard-prompt-injection** test split (241 injections). False alarms were measured on those sets' benign rows and on **240 of our own texts**: 60 each of vault notes, ticket notes, Finnish UI strings from the tendering monolith, and repo READMEs. A model flags a text when its score reaches the threshold, 0.9 unless stated.

Injections hidden inside tool output were measured on **AgentInjectionBench** (Apache-2.0, pinned revision ef230359): 142 cases where the attack sits in a tool result, MCP response, API response, RAG document or file, and 40 benign controls written to look attack-adjacent. Each case was scored on the text its tools returned, which is what the hook sees.

NVIDIA's indirect-injection set was dropped from the comparison: Horizon-Labs' published build scripts train on all of it, and on deepset's train split, so scores there measure memory. Prompt Guard 2 was scored from Gravitee's ungated ONNX build, because Meta's own is gated.

CLASSIFIERS

Horizon-Labs catches three to five times more than Prompt Guard 2

Both run as ONNX models with no PyTorch. Horizon-Labs is the only one that catches most attacks; Prompt Guard 2 almost never fires, and its quantized build loses about half of what the full-precision one catches.

		CAUGHT		FALSE ALARMS				PER TEXT	
MODEL	THRESHOLD	DEEPSET	XTRAM1	XTRAM1	VAULT	TICKETS	FINNISH UI	README S	MEDIAN	P95
Horizon-Labs prompt-injection-guard-base LIVE ModernBERT, 308M, 641 MB quantized, Apache-2.0	0.9	38.3%	55.6%	0.4%	3.3%	3.3%	0%	3.3%	14 ms	63 ms
Horizon-Labs, looser threshold	0.5	65.0%	80.5%	0.7%	6.7%	10.0%	0%	3.3%	14 ms	63 ms
Prompt Guard 2 86M, full precision 1.1 GB, Llama 4 licence	0.9	13.3%	42.7%	0.2%	0%	0%	0%	1.7%	29 ms*	169 ms*
Prompt Guard 2 86M, quantized 281 MB	0.9	5.0%	27.0%	0%	0%	0%	0%	0%	15 ms*	92 ms*

False alarms on xTRam1 are its 559 benign rows; deepset's 56 benign rows raised none from any classifier. * Timed on the Windows VM; Horizon-Labs was timed on the Mac, where it runs live (45 ms median on the VM). ProtectAI's deberta-v3 v2 was downloaded but not scored: it is archived, English only and stopped 22% of attacks in Meta's AgentDojo test.

DECISION MODELS

tev1:4b is the first one quiet enough to flag alone; laya:en flags over a quarter of our tickets

These are free "System 1" decision models in the style of TypeSafe Jev, asked the same two questions from laya's guard preset (injection, jailbreak). The Ollaya models read our tickets and notes, which are full of instructions written for people, as attacks; none of them is usable alone. **tev1:4b**, served by Ollama 0.35's `/v1/systemone`, is the exception: at 0.5 it catches more than laya:en does at 0.9 and wrongly flags 2 of our 240 texts. Its scores rarely reach 0.9, so it is read at 0.5.

	CAUGHT		FALSE ALARMS					PER TEXT	
MODEL	DEEPSET	XTRAM1	XTRAM1	VAULT	TICKETS	FINNISH UI	READMES	MEDIAN	P95
laya:en LIVE, AS SCREEN ModernBERT-large, 421M, 853 MB, Apple GPU	41.7%	**85.5%**	6.3%	**45.0%**	**28.3%**	**33.3%**	**20.0%**	39 ms	123 ms
laya:multilingual mmBERT, 322M, 683 MB	21.7%	56.8%	2.9%	**20.0%**	**56.7%**	**16.7%**	**31.7%**	15 ms	47 ms
qwen3guard 0.6B, fixed safety questions; jailbreak category used	0%	10.4%	0%	0%	0%	0%	0%	484 ms	882 ms
nli zero-shot DeBERTa, ~400M	0%	0.8%	0%	0%	0%	0%	0%	98 ms	426 ms
tev1:4b, at 0.5 Together AI, Qwen3.5 fine-tune, 4.5 GB, Ollama, Apple GPU	66.7%	**93.8%**	0.7%	0%	0%	0%	3.3%	433 ms	1,340 ms
nimble:9b, at 0.5 Bespoke Labs, 9B, 9.5 GB, Ollama, Apple GPU	51.7%	79.7%	0.4%	0%	0%	0%	0%	**1,449 ms**	**3,019 ms**
tev1:0.8b, at 0.5 Together AI, 811 MB, Ollama	78.3%	89.6%	**35.8%**	**96.7%**	**93.3%**	**80.0%**	**75.0%**	111 ms	497 ms

Four larger Ollaya models were scored only as second opinions, on the 84 texts Horizon-Labs was unsure of, because each call costs seconds on this Mac: jevk5 255 ms median (713 ms p95), winnow:e4b 363 ms (707 ms, Apple GPU), kev 1.4 s (11.3 s), decider:4b 2.9 s (11.6 s, CPU at full precision). At 0.9, tev1:4b catches 1.7% and 36.1%, nimble:9b 18.3% and 43.6%, and tev1:0.8b 0% and 10.8%. Inside tool output (AgentInjectionBench), alone: laya:en at 0.9 catches 28.2% with 2.5% false alarms, tev1:4b at 0.5 30.3% with none, nimble:9b 21.8% with none. Of deepset's 56 benign rows, laya:en, tev1:4b and nimble:9b flagged one each, tev1:0.8b 20, the others none.

COMBINATIONS

Without Claude, Horizon-Labs or tev1:4b catches the most

In a cascade, Horizon-Labs flags alone above 0.9 and hands its unsure band (0.5 to 0.9) to a second model, which flags at 0.9 (tev1:4b at 0.5). Asking laya:en this way adds 10 to 20 points of catch at the same false-alarm rate on tickets; tev1:4b adds 3 to 13 more. Flagging whenever any model fires floods our own text with alarms, except with tev1:4b, which is quiet enough to flag on its own.

	CAUGHT		FALSE ALARMS				
SETUP	DEEPSET	XTRAM1	XTRAM1	VAULT	TICKETS	FINNISH UI	README S
Horizon-Labs alone	38.3%	55.6%	0.4%	3.3%	3.3%	0%	3.3%
Horizon-Labs → laya:en the live setup until 2026-09-29 22:05	48.3%	75.5%	0.4%	6.7%	3.3%	0%	3.3%
Horizon-Labs → tev1:4b, at 0.5	61.7%	78.8%	0.4%	3.3%	3.3%	0%	3.3%
Horizon-Labs at 0.9 or tev1:4b at 0.5	75.0%	**95.0%**	0.9%	3.3%	3.3%	0%	6.7%
Horizon-Labs → laya:en or winnow:e4b	50.0%	78.4%	0.4%	6.7%	3.3%	0%	3.3%
Horizon-Labs → winnow:e4b	41.7%	67.2%	0.4%	3.3%	3.3%	0%	3.3%
Horizon-Labs → laya:multilingual	41.7%	64.3%	0.4%	5.0%	5.0%	0%	3.3%
Horizon-Labs → laya:en and winnow:e4b	40.0%	64.3%	0.4%	3.3%	3.3%	0%	3.3%
Horizon-Labs → decider:4b	38.3%	60.2%				same as Horizon-Labs alone	
Horizon-Labs → qwen3guard, jevk5, kev or nli	38.3%	55.6–58.1%				same as Horizon-Labs alone	
Any of Horizon-Labs, laya:en, laya:multilingual	65.0%	92.5%	8.1%	**53.3%**	**63.3%**	**43.3%**	**46.7%**
At least 2 of those 3	28.3%	69.3%	1.4%	**15.0%**	**25.0%**	6.7%	8.3%
Mean score of those 3	10.0%	47.3%	0%	0%	0%	0%	0%

False alarms on xTRam1 are its 559 benign rows; of deepset's 56, only "any of" and "Horizon-Labs or tev1:4b" flagged one. Inside tool output the two tev1:4b setups catch 66.9% and 69.0%, with Horizon-Labs' own 15% false alarms.

CLAUDE JUDGE

Sonnet without thinking is both the most accurate and the fastest judge

Claude reads only what the cascade flags and has the last word in the note; it never removes a warning, because it has read the suspect text itself. Judged here: 30 injections and all 10 benign texts among the 221 the live cascade flags. Thinking decides the speed: with it, Haiku wrote 650 to 1,150 tokens before a one-line answer.

JUDGE	KEEPS INJECTIONS	CLEARS FALSE ALARMS	MEDIAN	SLOWEST	VERDICT
Claude Sonnet, thinking off LIVE	**30 / 30**	10 / 10	1.9 s	2.8 s	Best on both counts
Claude Haiku, thinking on	28 / 30	10 / 10	5.1 s	**24.7 s**	Accurate, too slow
Claude Haiku, thinking off, sharper prompt	27 / 30	10 / 10	2.2 s	6.6 s	Misses 1 in 10
Claude Haiku, thinking off	**23 / 30**	10 / 10	2.2 s	3.9 s	Misses 1 in 4

The judge runs as `claude -p --model sonnet --thinking disabled` with no tools, settings sources, MCP servers, skills or saved session, its own empty config and working folders, and telemetry and update checks off. `--bare` would strip more but refuses subscription logins. Usage counts against the Claude plan; at pay-per-use prices a judgement is about $0.002.

LETTING CLAUDE DECIDE

Claude on everything either model finds suspect catches the most

Instead of only annotating flags, Claude can decide the grey zone: whatever Horizon-Labs scores 0.5 to 0.9, and, with laya:en as a screen that never flags by itself, whatever laya:en scores 0.5 or more. Claude kept every injection it was shown in these runs and cleared 26 of 27 benign texts, so it adds catches without adding noise. The price is how much ordinary text waits for it.

	CAUGHT			FALSE ALARMS		
SETUP	DEEPSET	XTRAM1	TOOL OUTPUT	OUR TEXTS	TOOL-OUTPUT CONTROLS	OUR TEXTS SENT TO CLAUDE
Horizon-Labs alone, 0.9	38.3%	55.6%	64.8%	2.5%	15%	–
Horizon-Labs → laya:en as confirmer	48.3%	75.5%	66.9%	3.3%	15%	3%
Horizon-Labs 0.5 or more → Claude	65.0%	80.5%	69.7%	2.5%	15%	5%
Horizon-Labs or laya:en 0.9 or more → Claude	78.3%	92.9%	73.2%	2.9%	15%	35%
Horizon-Labs or laya:en 0.5 or more → Claude LIVE	83.3%	**93.8%**	**78.9%**	2.9%	15%	**40%**

"Tool output" is AgentInjectionBench. Its 6 false alarms of 40 are the same in every setup and all come from Horizon-Labs at 0.9 or more; Claude clears 5, so 1 reaches the user as a notification. No laya:en threshold makes a cheaper middle: it scores most ordinary text near 1.0, so even at 0.999 it sends 23% of our texts to Claude.

OLLAMA DECISION MODELS · 2026-10-02

tev1:4b as the screen would cut Claude's load to a seventh, and miss more inside tool output

Ollama 0.35 serves decision models at `/v1/systemone`, with the same request as Ollaya. Swapping laya:en for one of them in the live setup keeps everything else equal: Horizon-Labs at 0.9 flags alone, and whatever Horizon-Labs or the screen scores 0.5 or more goes to Claude Sonnet, whose verdicts are reused from the runs above. **laya:en stays the screen**: it catches 7 more points of the injections hidden in tool output, which is what the hook exists for. tev1:4b is the better second opinion on a server without a Claude judge.

	CAUGHT			FALSE ALARMS			SCREEN, PER TEXT	
SCREEN	DEEPSET	XTRAM1	TOOL OUTPUT	OUR TEXTS	TOOL-OUTPUT CONTROLS	OUR TEXTS SENT TO CLAUDE	MEDIAN	P95
laya:en LIVE	**83.3%**	93.8%	**78.9%**	2.9%	15%	**40%**	**39 ms**	123 ms
tev1:4b tool output: 1.1 s median, 2.2 s p95	78.3%	**96.3%**	71.8%	2.5%	15%	**6%**	433 ms	1,340 ms
nimble:9b tool output: 1.5 s median, 1.8 s p95	76.7%	91.7%	70.4%	2.5%	15%	**5%**	**1,449 ms**	**3,019 ms**
tev1:0.8b	scores almost every text 0.5 to 0.9					**87%**	111 ms	497 ms

Claude judged 21 texts tev1:4b sends that laya:en never did; every other verdict was reused. The trade in waiting is close: laya:en costs every outside text about 40 ms and sends 40% of them on for about 2 s, tev1:4b costs every one about 430 ms and sends 6% on. Both tev1 sizes refuse a prompt over 2,050 tokens instead of truncating it, so a long outside text has to be split into several calls; nimble:9b reads 256K tokens but took 6.8 s on a 35,000-character text.

SPEED IN USE

What a tool call pays for the check

Timed through the real hook with the live configuration, median of five runs on the Windows VM. Most of the fixed cost is Python starting; only outside text travels to the Mac, and only flagged text waits for Claude.

TOOL CALL	TIME	WHAT RUNS
Local command output	124 ms	Secret redaction and patterns only
Outgoing Jira comment	119 ms	Secret and style checks before sending
Outside text, short and harmless	255 ms	Plus Horizon-Labs and laya:en on the Mac
Outside text, 38,561 characters	508 ms	Horizon-Labs over overlapping windows
Outside text Claude has to decide	1.7–3.8 s	Plus Claude Sonnet, and the desktop toast (about 0.5 s) when confirmed

FOOTPRINT

What stays installed

On the Mac: Horizon-Labs (675 MB with its tokenizer), laya:en in Ollaya (853 MB) and a Python environment with ONNX Runtime (about 160 MB). The trial pulled 32 GB of Ollaya models; everything but laya:en was removed afterwards. Two login items start Ollaya and the scoring server and restart them if they stop. Nothing on the Windows side needs PyTorch. The 2026-10-02 trial pulled tev1:0.8b, tev1:4b and nimble:9b into Ollama (14.8 GB) and removed them afterwards; the live setup does not use Ollama.

LIMITS OF THESE NUMBERS

Read small differences as noise

- **Small samples.** Each of our benign sources has 60 texts, so 3.3% is two texts; the judge was tested on 40.
- **Mostly direct, English attacks.** The one set of injections hidden inside tool output, AgentInjectionBench, has 182 cases, so read its numbers as direction, not precision. No Finnish attacks were tested; Finnish text raised no false alarm from Horizon-Labs.
- **Young software.** Horizon-Labs v2, laya and Ollaya were between six and eleven days old when measured, tev1 and nimble about a day old, and their publishers' own benchmarks are unaudited. Together AI notes it has not fully tested tev1 against prompt injection.
- **A tripwire, not a boundary.** A detected injection becomes a note and, when confirmed, a notification; nothing is blocked on it. Outgoing secrets and personal IDs are blocked.
- **Long texts.** laya:en reads about 512 tokens, so as a screen it sees the start of a long ticket; Horizon-Labs reads the whole text in overlapping windows. tev1 rejects anything over 2,050 tokens, so the benchmark splits such a text in halves and keeps the higher score; only 6 of the 1,338 samples are over 4,000 characters.
- **Claude is the linchpin.** If it cannot run (an expired token, say), the grey zone gets no verdict and only Horizon-Labs' 0.9 flags remain.